Language

Автор сайта

Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Dagger 2
сделайте код чище
курс от автора сайта
StartAndroid
MADE WITH MYBANNERMAKER.COM

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid
15 086 подписчиков
Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 24. LayoutModifier. Свои Modifier операторы. Chain.

18 июня 2023

В этом уроке используем LayoutModifier, чтобы создать свои простые версии операторов width, offset и padding. Разбираемся, как общаются между собой Modifier операторы в цепочке.

В прошлом уроке мы подробно разобрали как работает LayoutModifier. Теперь мы можем использовать его, чтобы создавать простые аналоги существующих Modifier операторов.

width

Назовем наш оператор - myWidth:

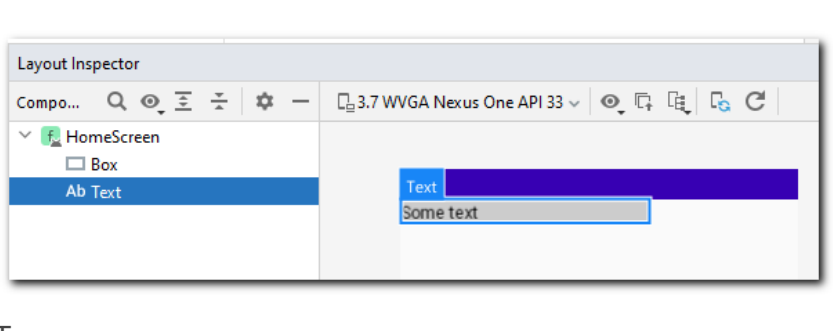
```
fun Modifier.myWidth(width: Dp) = layout { measurable, constraints ->
    val myConstraints = constraints.copy(
        minWidth = width.roundToPx(),
        maxWidth = width.roundToPx()
    )
    val placeable = measurable.measure(myConstraints)
    layout(placeable.width, placeable.height) {
        placeable.placeRelative(0, 0)
    }
}
```

На вход мы получаем желаемое значение ширины - width. Мы помещаем его в constraints, который пришел к нам от Box, и передаем в Text. Теперь Text должен будет подстроиться под ширину, которую мы указали. Больше от нас ничего не требуется. Остальной код - стандартный.

Используем наш новый оператор в Text:

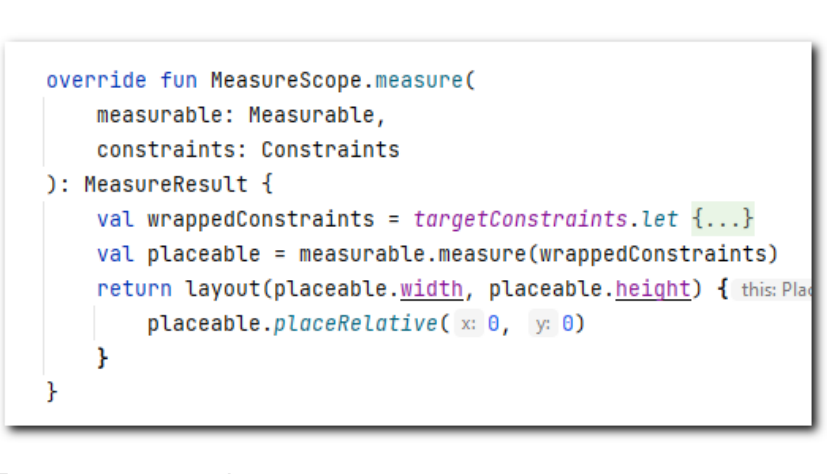
```
@Composable
fun HomeScreen() {
    Box(
        modifier = Modifier.background(Color.LightGray)
    ) {
        Text(text = "Some text", modifier = Modifier.myWidth(200.dp))
    }
}
```

Результат



Текст занимает нужную нам ширину

В простых случаях наш оператор работает неплохо. Но стандартный Modifier.width учитывает кучу всяких нюансов. Вы можете открыть его исходники и посмотреть, что там выполняется множество проверок. А в конце можно найти код, который похож на наш:



```
override fun MeasureScope.measure(
    measurable: Measurable,
    constraints: Constraints
): MeasureResult {
    val wrappedConstraints = targetConstraints.let {...}
    val placeable = measurable.measure(wrappedConstraints)
    return layout(placeable.width, placeable.height) { this: Plac
        placeable.placeRelative( x: 0,  y: 0)
    }
}
```

Пересчет constraints и передача их дочернему элементу.

offset

Создадим myOffsetX, который будет выполнять сдвиг по горизонтали:

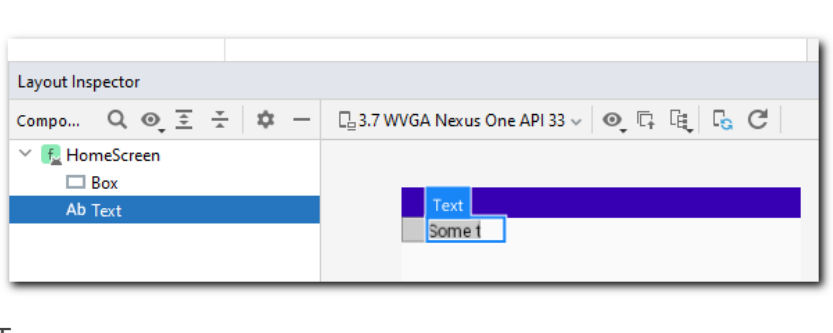
```
fun Modifier.myOffsetX(offset: Dp) = layout { measurable, constraints ->
    val placeable = measurable.measure(constraints)
    layout(placeable.width, placeable.height) {
        placeable.placeRelative(offset.roundToPx(), 0)
    }
}
```

Тут все еще проще, мы уже делали подобное в прошлом уроке. Просто используем значение offset в placeRelative, чтобы сдвинуть дочерний элемент по оси X.

Используем новый оператор в нашем примере:

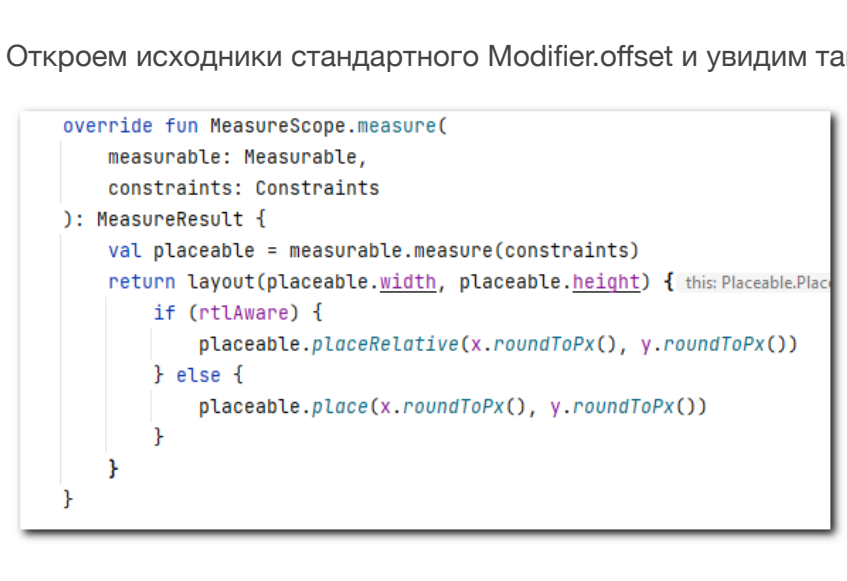
```
@Composable
fun HomeScreen() {
    Box(
        modifier = Modifier.background(Color.LightGray)
    ) {
        Text(text = "Some text", modifier = Modifier.myOffsetX(20.dp))
    }
}
```

Результат:



Текст сдвинулся.

Откроем исходники стандартного Modifier.offset и увидим там примерно то же самое, что и у нас:



```
override fun MeasureScope.measure(
    measurable: Measurable,
    constraints: Constraints
): MeasureResult {
    val placeable = measurable.measure(constraints)
    return layout(placeable.width, placeable.height) { this: Placeable.Plac
        if (rtlAware) {
            placeable.placeRelative(x.roundToPx(), y.roundToPx())
        } else {
            placeable.place(x.roundToPx(), y.roundToPx())
        }
    }
}
```

padding

Создадим оператор myPaddingStart, который будет добавлять padding слева.

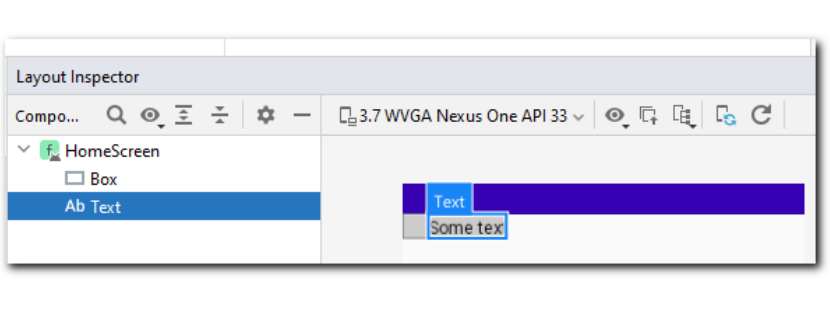
```
fun Modifier.myPaddingStart(padding: Dp) = layout { measurable, constraints ->
    val placeable = measurable.measure(constraints)
    layout(placeable.width + padding.roundToPx(), placeable.height) {
        placeable.placeRelative(padding.roundToPx(), 0)
    }
}
```

Код похож на myOffsetX - он тоже делает сдвиг в placeRelative. Но кроме этого, он увеличивает свой размер на величину сдвига. Таким образом, он передаст в Box правильный размер с учетом сдвига, и Text не будет обрезан.

Используем в примере:

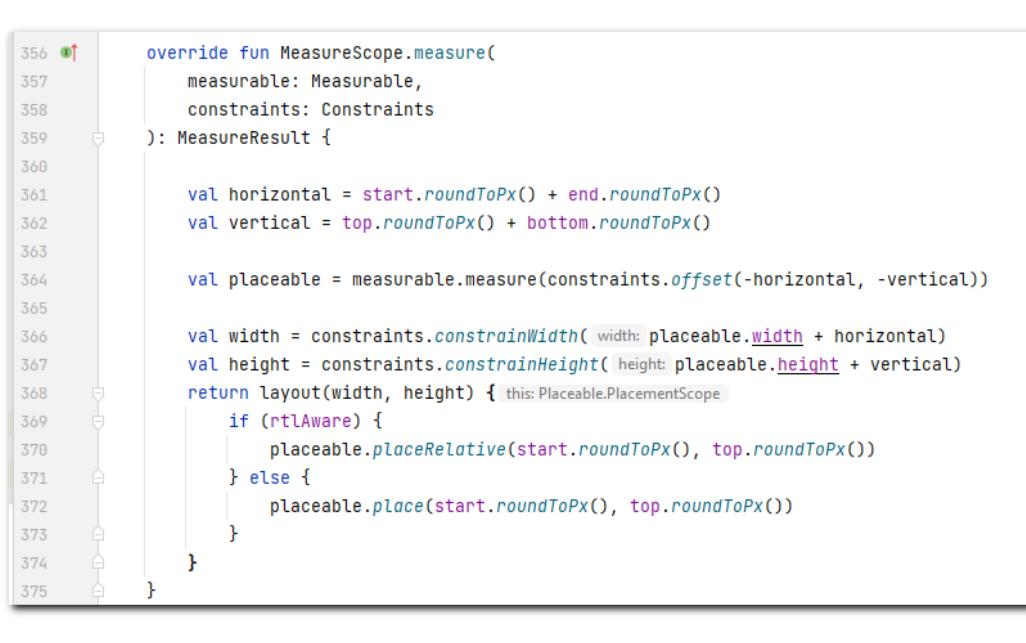
```
@Composable
fun HomeScreen() {
    Box(
        modifier = Modifier.background(Color.LightGray)
    ) {
        Text(text = "Some text", modifier = Modifier.myPaddingStart(20.dp))
    }
}
```

Результат:



Box получил от myPaddingStart (в отличие от myOffsetX) увеличенные размеры, которые включают в себя сдвиг. Поэтому он больше не обрезает Text.

В исходниках Modifier.padding примерно та же логика:



```
override fun MeasureScope.measure(
    measurable: Measurable,
    constraints: Constraints
): MeasureResult {

    val horizontal = start.roundToPx() + end.roundToPx()
    val vertical = top.roundToPx() + bottom.roundToPx()

    val placeable = measurable.measure(constraints.offset(-horizontal, -vertical))

    val width = constraints.constrainWidth( width: placeable.width + horizontal)
    val height = constraints.constrainHeight( height: placeable.height + vertical)
    return layout(width, height) { this: Placeable.PlacementScope
        if (rtlAware) {
            placeable.placeRelative(start.roundToPx(), top.roundToPx())
        } else {
            placeable.place(start.roundToPx(), top.roundToPx())
        }
    }
}
```

В placeRelative он делает сдвиг на start и top. А в layout передает размеры дочернего элемента с учетом сдвига.

Основное отличие от нашей реализации - constraints. В measurable.measure он передает constraints из которого вычитает значение padding. Эту логику можно легко понять на примере.

Допустим у нас в constraints пришла ширина 200, а значение горизонтального padding = 16. Это приводит к тому, что итоговая доступная ширина для дочернего элемента = 200 - 16 (слева) - 16 (справа) = 168. Именно такую ширину в constraints мы и должны передавать дочернему элементу.

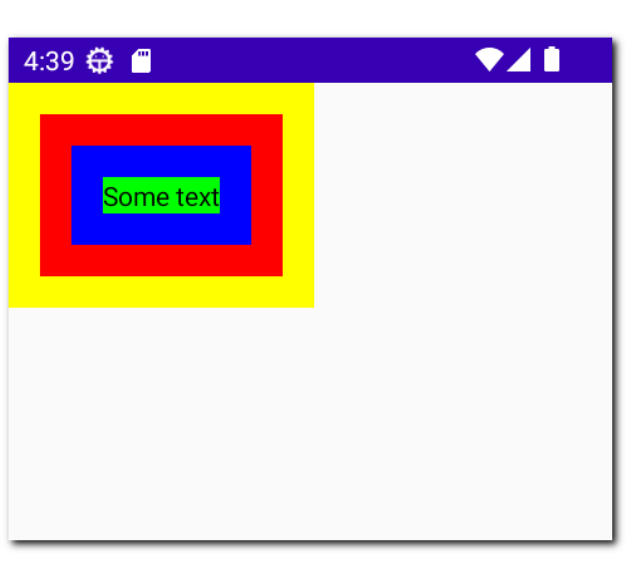
Modifier chain

В уроках про Modifier мы говорили о том, что можно применять один и тот же оператор несколько раз, и что важен порядок операторов.

Если, например, мы чередуем padding и background:

```
@Composable
fun HomeScreen() {
    Box(
        modifier = Modifier.background(Color.LightGray)
    ) {
        Text(
            text = "Some text", modifier = Modifier
                .background(Color.Yellow)
                .padding(25.px)
                .background(Color.Red)
                .padding(25.px)
                .background(Color.Blue)
                .padding(25.px)
                .background(Color.Green)
        )
    }
}
```

то получим такой результат:



Код расширения px с прошлого урока:

```
inline val Int.px: Dp
    @Composable get() = with(LocalDensity.current) { this@px.toDp() }
```

Знание LayoutModifier дает нам возможность увидеть, как это работает под капотом. Создадим свой простой аналог padding:

```
fun Modifier.myPadding(id: Int) = layout { measurable, constraints ->
    Log.d(TAG, "myPadding $id, measure child")
    val placeable = measurable.measure(constraints)

    val myWidth = placeable.width + 50
    val myHeight = placeable.height + 50
    Log.d(TAG, "myPadding $id, child: (${placeable.width}, ${placeable.height}), me: ($myWidth, $myH

    layout(myWidth, myHeight) {
        placeable.placeRelative(25, 25)
    }
}
```

Он добавляет 50 px к своей ширине и высоте, и смещает позицию дочернего элемента на 25 px. Таким образом дочерний элемент будет окружен отступом в 25 px с каждой стороны.

На вход myPadding принимает id. Это нужно для логирования. Мы будем использовать несколько myPadding в одной цепочке, и нам надо будет отличать их логи. В лог мы пишем, что myPadding собирается запросить размер у дочернего элемента - measure child. После получения результата мы выводим в лог размеры дочернего элемента и планируемые размеры текущего myPadding.

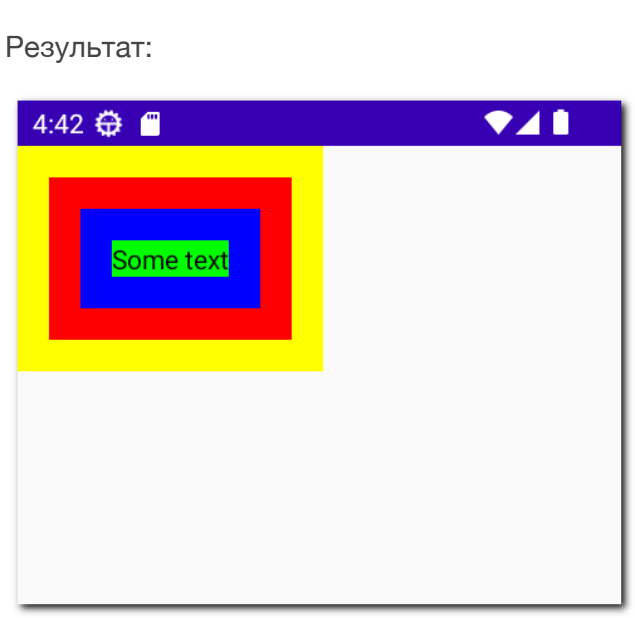
В примере выше заменим padding на myPadding:

```
@Composable
fun HomeScreen() {
    Box(
        modifier = Modifier.background(Color.LightGray)
    ) {
        Text(text = "Some text", modifier = Modifier
            .background(Color.Yellow)
            .myPadding(1)
            .background(Color.Red)
            .myPadding(2)
            .background(Color.Blue)
            .myPadding(3)
            .background(Color.Green)
        )
    }
}
```

Эта цепочка Modifier операторов построит нам следующую Layout иерархию:
Box > myPadding1 > myPadding2 > myPadding3 > Text

background - это не LayoutModifier, а DrawModifier. Он отвечает не за Layout фазу, а за Draw. Поэтому он никак не участвует в Layout делах по измерению размеров и определению позиций.

Результат:



Смотрим логи:
myPadding 1, measure child
myPadding 2, measure child
myPadding 3, measure child
myPadding 3, child: (93, 29), me: (143, 79)
myPadding 2, child: (143, 79), me: (193, 129)
myPadding 1, child: (193, 129), me: (243, 179)

Для лучшего понимания давайте дополним их, как будто Box и Text тоже вели свои логи:
Box, measure child
myPadding 1, measure child
myPadding 2, measure child
myPadding 3, measure child
Text, no child, me: (93, 29)
myPadding 3, child: (93, 29), me: (143, 79)
myPadding 2, child: (143, 79), me: (193, 129)
myPadding 1, child: (193, 129), me: (243, 179)
Box, child: (243, 179), me: (243, 179)

Мы видим, как запрос размера (measure child) идет вниз по Layout иерархии:
Box > myPadding1 > myPadding2 > myPadding3 > Text

Каждый дочерний элемент запрашивает размер у своего дочернего элемента прежде, чем определиться со своим размером.

В итоге дело доходит до Text. Он вычисляет свой размер и отправляет родителю - myPadding 3. Тот добавляет к размеру 50, и отправляет своему родителю - myPadding 2. Тот снова добавляет 50 и отправляет своему родителю - myPadding 1. Тот снова добавляет 50 и отправляет своему родителю - Box.

В итоге мы получаем такую Layout иерархию:

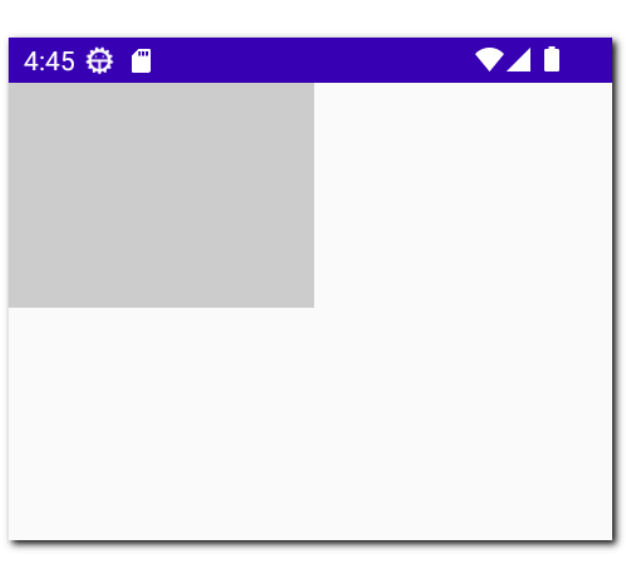
Box (w = 243, h = 179, x = 0, y = 0)
 myPadding1 (w = 243, h = 179, x = 0, y = 0)
  myPadding2 (w = 193, h = 129, x = 25, y = 25)
   myPadding3 (w = 143, h = 79, x = 25, y = 25)
    Text (w = 93, h = 29, x = 25, y = 25)

А если смотреть с точки зрения не только Layout, а всех Modifier операторов, то получится так:

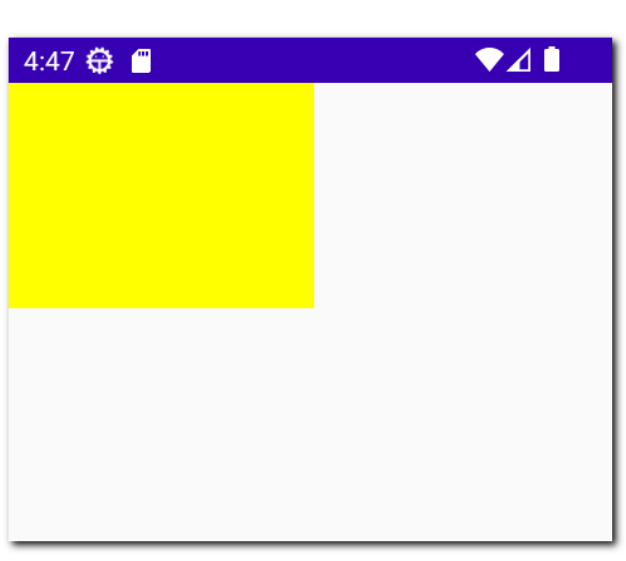
background(Color.LightGray)
Box (w = 243, h = 179, x = 0, y = 0)
 background(Color.Yellow)
 myPadding1 (w = 243, h = 179, x = 0, y = 0)
  background(Color.Red)
  myPadding2 (w = 193, h = 129, x = 25, y = 25)
   background(Color.Blue)
   myPadding3 (w = 143, h = 79, x = 25, y = 25)
    background(Color.Green)
    Text (w = 93, h = 29, x = 25, y = 25)

Я попытался по исходникам background понять, как он работает, но сходу не получилось. Но видно, что background для рисования использует координаты ближайшего по цепочке LayoutModifier.

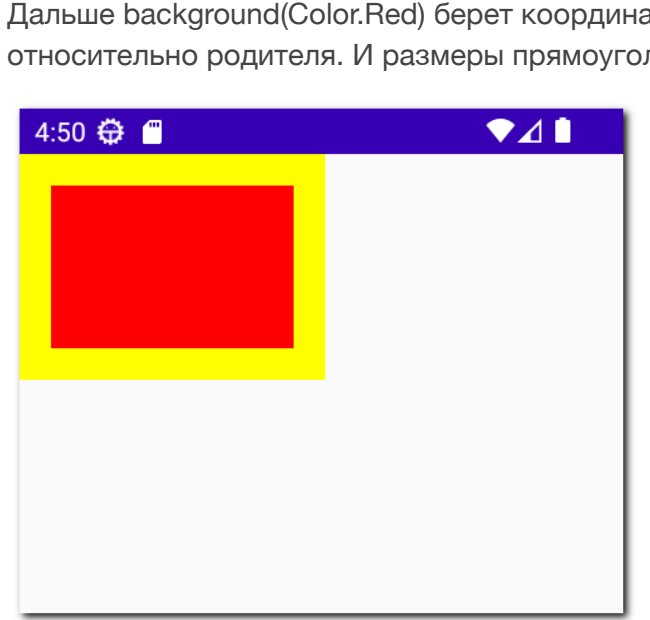
В нашем примере background(Color.LightGray) берет размер и координаты из Box (w = 243, h = 179, x = 0, y = 0) и рисует по ним серый прямоугольник.



Но мы на экране его не видим, потому что background(Color.Yellow) получает от myPadding1 (w = 243, h = 179, x = 0, y = 0) точно такие же размер и координаты и полностью закрашивает серый прямоугольник желтым.

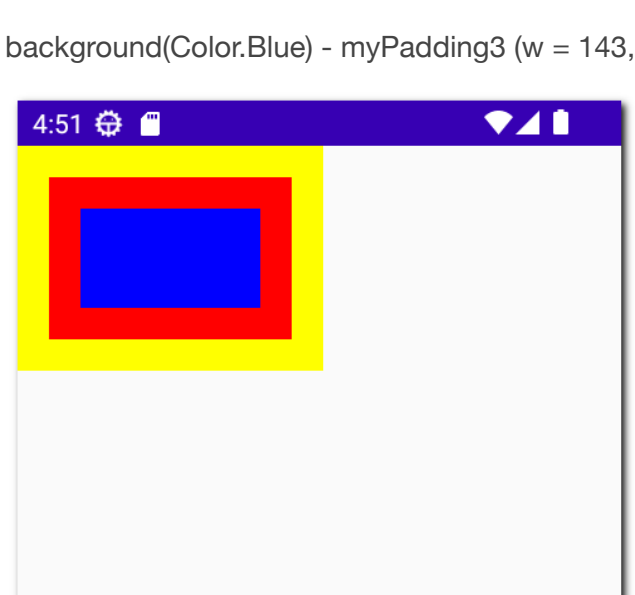


Дальше background(Color.Red) берет координаты myPadding2 (w = 193, h = 129, x = 25, y = 25). Тут уже есть смещение на 25x25 относительно родителя. И размеры прямоугольника меньше. Именно это мы и видим на экране.

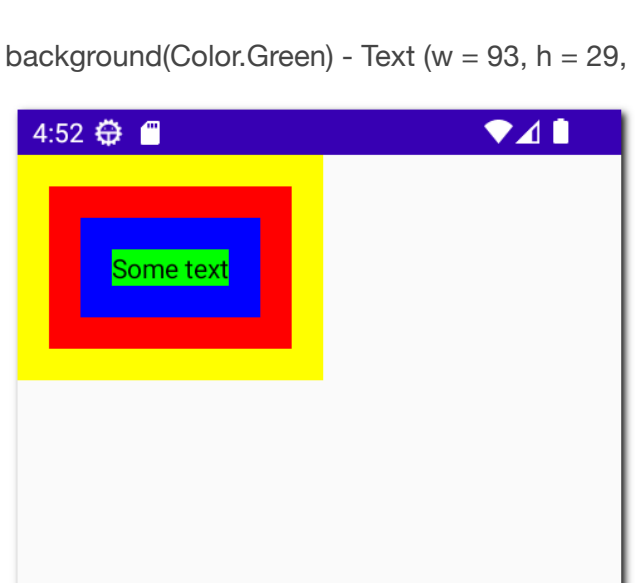


Ну и далее аналогично идут пары:

background(Color.Blue) - myPadding3 (w = 143, h = 79, x = 25, y = 25)



background(Color.Green) - Text (w = 93, h = 29, x = 25, y = 25)



Они рисуют синий и зеленый прямоугольники, которые еще меньше и тоже смещены на 25x25 относительно своих родителей. А в самом конце Text рисует себя.

Все это не очень просто для понимания, зато дает представление о том, как работает последовательность LayoutModifier операторов.

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Комментарии

# **Путаница с цепочкой методов у Modifier** — **Александр** 27.10.2023 13:43
Здравствуйте, большое спасибо за данный урок, действительно очень интересно, но в голове возникла небольшая путаница по поводу того в какой последовательности применяются методы myPadding и background, так как в общем логе последовательность такая: measure1 -> measure2 -> measure3, newSize3 -> newSize2 -> newSize1, я повторил данный пример и убедился что это действительно так, но далее вы объясняете в той последовательности, как-будто последовательность применения свойств: yellow-> myPadding1 -> red-> myPadding2 -> blue -> myPadding3 -> green, но на деле, как я понял, если рассматривать всю последовательность применения будет так: green-> myPadding3(+50) -> blue -> myPadding2(+50) -> red -> myPadding1(+50) -> yellow. Получается что у нас есть стек свойств, который формируется в порядке 1 -> 2 -> 3 и возвращает результаты в порядке 3 -> 2 -> 1
Ответить

# **RE: Путаница с цепочкой методов у Modifier** — **Dmitry Vinogradov** 20.11.2023 18:51
Про цвета - это уже не последовательность действий, а просто откуда берутся размеры прямоугольников. Последовательность там будет все та же: 1>2>3>2>1
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.